P1 — Square Areas and Binary Search

```
#include <stdio.h>
int main() {
    double a[10], key;
    int i, low = 0, high = 9, mid, found = -1;
    a[0] = 32 * 32;
    for (i = 1; i < 10; i++) a[i] = a[i - 1] / 2;
    for (i = 0; i < 10; i++)
        printf("SQ-%d: %.2f\n", i + 1, a[i]);
    printf("Enter area to search: ");
    scanf("%lf", &key);
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) { found = mid; break; }
        else if (key > a[mid]) high = mid - 1;
        else low = mid + 1;
    }
    if (found != -1)
        printf("Found in SQ-%d\n", found + 1);
    else printf("Area not found\n");
    return 0;
}
```

P2A — Matrix Multiplication and Tri-Diagonals

```
#include <stdio.h>
int main() {
    int A[4][2], B[2][4], C[4][4] = {0};
    int i, j, k;
    printf("Enter 8 Dalta Gang amounts:\n");
    for (i = 0; i < 4; i++)
        for (j = 0; j < 2; j++) scanf("%d", &A[i][j]);
    printf("Enter 8 Malta Gang amounts:\n");
    for (i = 0; i < 2; i++)
        for (j = 0; j < 4; j++) scanf("%d", &B[i][j]);
    printf("Dalta Gang (4x2):\n");
    for (i = 0; i < 4; i++) {
        for (j = 0; j < 2; j++) printf("%d ", A[i][j]);
        printf("\n");
    }
    printf("Malta Gang (2x4):\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 4; j++) printf("%d ", B[i][j]);
        printf("\n");
    }
    for (i = 0; i < 4; i++)
        for (j = 0; j < 4; j++)
            for (k = 0; k < 2; k++)
                C[i][j] += A[i][k] * B[k][j];
    printf("Result matrix (4x4):\n");
    for (i = 0; i < 4; i++) {
        for (j = 0; j < 4; j++) printf("%d ", C[i][j]);
        printf("\n");
    }
    printf("Main diagonal: ");
    for (i = 0; i < 4; i++) printf("%d ", C[i][i]);
    printf("\nUpper diagonal: ");
```

```
    for (i = 0; i < 3; i++) printf("%d ", C[i][i + 1]);
    printf("\nLower diagonal: ");
    for (i = 1; i < 4; i++) printf("%d ", C[i][i - 1]);
    printf("\n");
    return 0;
}
```

P2B — Tower of Hanoi

```
#include <stdio.h>
void hanoi(int n, char from, char aux, char to) {
    if (n == 0) return;
    hanoi(n - 1, from, to, aux);
    printf("Move disk %d from %c to %c\n", n, from, to);
    hanoi(n - 1, aux, from, to);
}
int main() {
    int n;
    printf("Enter number of disks: ");
    scanf("%d", &n);
    if (n < 1) { printf("Enter a positive number\n"); return 0; }
    hanoi(n, 'A', 'B', 'C');
    return 0;
}
```

P3 — Wallets Insertion Sort

```
#include <stdio.h>
int main() {
    int a[20], i, j, key;
    for (i = 0; i < 20; i++) a[i] = 20 + 10 * i;
    for (i = 1; i < 20; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
    printf("Sorted wallet amounts:\n");
    for (i = 0; i < 20; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

P4 — Treasure Chests Quickselect

```
#include <stdio.h>
void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}
int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1, j;
    for (j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
```

```
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}
int quickselect(int a[], int low, int high, int k) {
    while (low <= high) {
        int pos = partition(a, low, high);
        if (pos == k) return a[pos];
        if (pos > k) high = pos - 1;
        else low = pos + 1;
    }
    return -1; /* Unreachable for valid k */
}
int main() {
    int a[15], i, rank;
    printf("Enter 15 distinct treasure values:\n");
    for (i = 0; i < 15; i++) scanf("%d", &a[i]);
    printf("Enter rank (1=highest, 4=fourth highest): ");
    scanf("%d", &rank);
    if (rank < 1 || rank > 15) {
        printf("Invalid rank\n"); return 0;
    }
    printf("Rank %d highest value = %d\n", rank,
           quickselect(a, 0, 14, 15 - rank));
    return 0;
}
```

P5 — Linked List Cycle Detection

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int data; struct Node *next; } Node;
Node *head = NULL, *tail = NULL;
Node *add(int x) {
    Node *n = malloc(sizeof(Node));
    n->data = x; n->next = NULL;
    if (head == NULL) head = n;
    else tail->next = n;
    tail = n;
    return n;
}
int main() {
    Node *node5 = NULL, *node6 = NULL, *n, *slow, *fast;
    int i;
    for (i = 1; i <= 10; i++) {
        n = add(i);
        if (i == 5) node5 = n;
        if (i == 6) node6 = n;
    }
    n = malloc(sizeof(Node));
    n->data = 15;
    n->next = node6->next;
    node6->next = n;
    tail->next = node5;  /* create cycle: 10 -> 5 */
    slow = fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) break;
```

```
    }
    if (fast == NULL || fast->next == NULL)
        printf("No cycle found\n");
    else {
        slow = head;
        while (slow != fast) {
            slow = slow->next;
            fast = fast->next;
        }
        printf("Cycle found; starts at %d\n", slow->data);
    }
    return 0;
}
```

P6 — BST Insertion and Traversals

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int data; struct Node *left, *right; } Node;
Node *newNode(int x) {
    Node *n = malloc(sizeof(Node));
    n->data = x; n->left = n->right = NULL;
    return n;
}
Node *insert(Node *root, int x, int *count) {
    if (root == NULL) return newNode(x);
    if (count != NULL) (*count)++;
    if (x < root->data) root->left = insert(root->left, x, count);
    else if (x > root->data) root->right = insert(root->right, x, count);
    return root;
}
void inorder(Node *r) {
    if (r == NULL) return;
    inorder(r->left); printf("%d ", r->data); inorder(r->right);
}
void preorder(Node *r) {
    if (r == NULL) return;
    printf("%d ", r->data); preorder(r->left); preorder(r->right);
}
int main() {
    int a[] = {30,15,40,8,16,35,32,50,55};
    int i, count = 0;
    Node *root = NULL;
    for (i = 0; i < 9; i++) root = insert(root, a[i], NULL);
    printf("Inorder: "); inorder(root);
    root = insert(root, 20, &count);
    printf("\nComparisons for 20: %d", count);
    printf("\nUpdated preorder: "); preorder(root);
    return 0;
}
```

P7 — BST Deletion and Rotation

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int data; struct Node *left, *right; } Node;
Node *make(int x) {
    Node *n = malloc(sizeof(Node));
    n->data = x; n->left = n->right = NULL;
```

```
    return n;
}
Node *insert(Node *r, int x) {
    if (!r) return make(x);
    if (x < r->data) r->left = insert(r->left, x);
    else if (x > r->data) r->right = insert(r->right, x);
    return r;
}
Node *minimum(Node *r) {
    while (r->left) r = r->left;
    return r;
}
Node *deleteNode(Node *r, int x) {
    if (!r) return NULL;
    if (x < r->data) r->left = deleteNode(r->left, x);
    else if (x > r->data) r->right = deleteNode(r->right, x);
    else {
        if (!r->left) { Node *t = r->right; free(r); return t; }
        if (!r->right) { Node *t = r->left; free(r); return t; }
        Node *t = minimum(r->right);
        r->data = t->data;
        r->right = deleteNode(r->right, t->data);
    }
    return r;
}
Node *rotateAt50(Node *r) {
    if (!r) return NULL;
    if (r->data == 50 && r->right) {
        Node *y = r->right;
        r->right = y->left;
        y->left = r;
        return y;
    }
    r->left = rotateAt50(r->left);
    r->right = rotateAt50(r->right);
    return r;
}
void postorder(Node *r) {
    if (!r) return;
    postorder(r->left); postorder(r->right); printf("%d ", r->data);
}
int main() {
    int a[] = {30,15,40,8,20,35,32,50,16,55,60};
    int i;
    Node *root = NULL;
    for (i = 0; i < 11; i++) root = insert(root, a[i]);
    root = deleteNode(root, 16);
    root = rotateAt50(root);
    printf("Postorder: "); postorder(root);
    return 0;
}
```

P8 — Merge Sort and Bubble Sort

```
#include <stdio.h>
void merge(int a[], int l, int m, int r, int *count) {
    int t[10], i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        (*count)++;
        if (a[i] <= a[j]) t[k++] = a[i++];
        else t[k++] = a[j++];
```

```
    }
    while (i <= m) t[k++] = a[i++];
    while (j <= r) t[k++] = a[j++];
    for (i = l; i <= r; i++) a[i] = t[i];
}
void mergeSort(int a[], int l, int r, int *count) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m, count);
    mergeSort(a, m + 1, r, count);
    merge(a, l, m, r, count);
}
void bubbleSort(int a[], int n, int *count) {
    int i, j, t;
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - 1 - i; j++) {
            (*count)++;
            if (a[j] > a[j + 1]) {
                t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
        }
}
void printArray(int a[], int n) {
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
int main() {
    int original[10] = {16,8,30,45,90,12,3,77,23,60};
    int a[10], b[10], i, mc = 0, bc = 0;
    for (i = 0; i < 10; i++) a[i] = b[i] = original[i];
    mergeSort(a, 0, 9, &mc);
    bubbleSort(b, 10, &bc);
    printf("Merge Sort: "); printArray(a, 10);
    printf("Merge comparisons: %d\n", mc);
    printf("Bubble Sort: "); printArray(b, 10);
    printf("Bubble comparisons: %d\n", bc);
    return 0;
}
```